AegisFlow DevSecOps Framework

Mục tiêu: chứng minh một pipeline DevSecOps chạy local bằng Docker, có scan bảo mật, policy gate, dashboard và tài liệu nộp bài trong một repository duy nhất.

Phạm vi: Task 1-4 chạy trên cùng ứng dụng demo, cùng môi trường `docker-compose.yml`, cùng pipeline `pipeline/run_pipeline.sh`. Task 5 là threat model cho một hệ thống riêng theo đúng yêu cầu đề bài.

5	112	15	1
Task covered	**Total findings**	**Critical**	**Docker env**
Task 1, 2, 3, 4, 5	Theo report mẫu hiện tại	Policy đang BLOCK	Chạy local thống nhất

1. Mô hình hệ thống

```
Reviewer
  -> docker compose up --build
  -> AegisFlow Dashboard :58081
  -> server.py /api/scan
  -> pipeline/run_pipeline.sh
  -> Build -> Test -> SAST -> SCA -> SBOM -> Secret -> IaC -> DAST -> Report -> Policy
  -> security-results/*.json
  -> dashboard/data/*.json
  -> DevSecOps_CaseStudy_Report.pdf
```

Thành phần chính: `dashboard/` là giao diện xem kết quả; `server.py` nhận yêu cầu scan; `pipeline/` chứa từng scanner bridge; `security-results/` là nơi sinh artifact JSON; `real-apps/` chứa ứng dụng mẫu để scan.

2. Hướng dẫn chạy demo

1. Bật Docker Desktop, sau đó chạy `docker compose up --build` tại root repo.
2. Mở `http://localhost:58081`.
3. Chọn target, ví dụ `/app/workspace/juice-shop`, `/app/workspace/python-vuln-app` hoặc ứng dụng mẫu được mount trong `real-apps/`.
4. Nếu muốn DAST live, nhập URL như `http://juice-shop:3000`; nếu bỏ trống, pipeline ghi nhận chế độ DAST dự đoán và nêu giới hạn.
5. Nhấn scan, xem trạng thái pipeline, policy decision, SBOM, audit log và các report trong `security-results/`.

3. Mapping yêu cầu nộp bài

Yêu cầu	Artifact đáp ứng	Ghi chú
Repo public duy nhất	Toàn bộ source, app mẫu, dashboard, pipeline và report nằm trong repo này.	Task 1-4 dùng cùng target/pipeline/env.
`docker-compose.yml`	Service `aegisflow` và `juice-shop`.	Chạy lại bằng Docker, không phụ thuộc CI thật.
PDF 3-5 trang	`DevSecOps_CaseStudy_Report.pdf`.	Tài liệu chấm điểm chính.
Security outputs	`sast_results.json`, `sca_results.json`, `iac_results.json`, `sbom.json`, `policy_result.json`.	Report có thể được parse tự động.

Task 1. CI/CD Pipeline + SAST

Pipeline tối thiểu gồm **Build**, **Test**, **Security** và **Report**. Cấu hình CI tham chiếu nằm ở `.gitlab-ci.yml`; local runner nằm ở `pipeline/run_pipeline.sh`. Cách này cho phép người chấm chạy bằng Docker nhưng vẫn thấy được tư duy CI/CD thật.

Flow thực thi chi tiết

```
1. build_target.py      : phát hiện stack, validate build script hoặc chạy build thật khi có cấu hình
2. test_target.py       : chạy test thật nếu có lệnh portable; nếu không, ghi rõ validation mode
3. scan_sast.py         : Semgrep scan, xuất SARIF/JSON
4. scan_sca.py          : Trivy filesystem dependency scan
5. scan_sbom.py         : sinh SBOM để truy vết supply chain
6. scan_secret.py       : Gitleaks phát hiện secret hardcoded
7. scan_iac.py          : Checkov kiểm tra Docker/IaC
8. scan_dast.py         : Nuclei/DAST live hoặc predictive mode
9. report_generator.py  : hợp nhất report
10. ai_triage_engine.py : triage ngữ cảnh, fallback deterministic khi không có GROQ_API_KEY
11. policy_engine.py    : quyết định pass/warn/block
```

Lý do chọn SAST tool

Công cụ	Lý do chọn	Vị trí trong pipeline
Semgrep	Nhẹ, chạy tốt trong container, hỗ trợ nhiều ngôn ngữ, dễ xuất SARIF/JSON và phù hợp repo nhiều sample app.	Sau Build/Test, trước SCA/IaC/DAST để phát hiện lỗi code sớm.

Pass/Fail decision

Loại issue	Quyết định	Lý do
Critical Secret, RCE, auth bypass, SQL injection có khả năng khai thác cao	Block pipeline	Rủi ro trực tiếp tới dữ liệu, credential hoặc quyền truy cập.
High từ 2 findings trở lên	Block pipeline	Rủi ro tích lũy lớn, cần xử lý trước khi merge/release.
Medium code smell bảo mật, missing hardening	Warning + ticket	Không nên chặn mọi release, nhưng phải có SLA.
Low informational	Allow + backlog	Dùng cho cải tiến hygiene.

Kết quả mẫu từ report hiện tại

ID	Tool	Severity	Ví dụ finding	Vị trí
FIND-009	Semgrep	Medium	javascript.browser.security.eval-detected.eval-detected	`app/routes/contributions.js:32`
FIND-010	Semgrep	Medium	code-string-concat	`app/routes/contributions.js:32`

Giả lập và giới hạn

Build/Test có thể chạy ở **validation mode** khi target không có dependency hoặc command portable. Đây không phải build production đầy đủ; mục tiêu là chứng minh orchestration, artifact và policy flow. Khi đưa vào CI thật, bật cài dependency theo stack và chạy test suite đầy đủ.

Task 2. SCA + Container / IaC Security

Repo đáp ứng nhiều hơn yêu cầu tối thiểu: có SCA, SBOM, secrets scanning, IaC scanning và container-oriented validation. Trivy kiểm tra dependency/filesystem; Checkov kiểm tra Dockerfile, GitHub workflow và cấu hình hạ tầng; Syft sinh SBOM.

75
SCA
Dependency vulnerabilities

3
IaC
Misconfiguration

5
Secrets
Hardcoded credentials

Phân biệt nhóm rủi ro

Nhóm	Định nghĩa	Tool/Artifact	Fix ở đâu
Vulnerability do code	Lỗi nằm trong source do lập trình sai, ví dụ dùng `eval`, nối chuỗi vào query, thiếu kiểm tra auth.	Semgrep/SAST	Sửa code, thêm test bảo mật.
Vulnerability do dependency	Thư viện/package có CVE hoặc version lỗi thời.	Trivy/SCA, Syft/SBOM	Nâng version, thay dependency, pin version an toàn.
Misconfiguration	Cấu hình Docker, workflow, permission hoặc image thiếu hardening.	Checkov/IaC	Sửa Dockerfile, compose, CI workflow, service account.
Secret exposure	API key, private key hoặc token bị commit vào repo.	Gitleaks	Rotate secret, xóa khỏi git history, dùng secret manager.

Issue High/Critical - root cause và remediation

Finding	Root cause	Tác động	Remediation
Critical Hardcoded API key tại `config/env/development.js`	Credential được lưu trong source thay vì biến môi trường/secret store.	Lộ token, truy cập trái phép tới service nội bộ hoặc bên thứ ba.	Rotate key, xóa khỏi history, dùng `.env` ignored + CI secret variable, thêm secret scan pre-merge.
Critical Private key tại `artifacts/cert/server.key`	Private key test bị đưa vào repo.	Giả mạo TLS, giải mã traffic hoặc impersonation nếu key được dùng thật.	Thu hồi cert/key, tạo key mới, không commit private key, mount qua runtime secret.
Medium Dockerfile thiếu HEALTHCHECK	Container không có health signal chuẩn.	Deploy khó phát hiện service treo, ảnh hưởng availability.	Thêm `HEALTHCHECK` phù hợp endpoint readiness.

Container / IaC logic

Pipeline không chỉ tìm CVE mà còn kiểm tra "đường đi tới production": Dockerfile có healthcheck hay không, workflow có permission quá rộng hay không, secret có bị lộ trước khi build image hay không. Với container image scanning production, bước tiếp theo là thêm `trivy image` sau khi build image thay vì chỉ `trivy fs`.

Hướng dẫn chạy riêng Task 2

1. Chạy toàn bộ stack bằng `docker compose up --build`.
2. Trong dashboard, chọn target và launch scan.
3. Xem `security-results/sca_results.json`, `security-results/iac_results.json`, `security-results/sbom.json`.
4. Đối chiếu bảng remediation trên dashboard hoặc trong full report.

Task 3. Policy Enforcement & Security Governance

Scenario đề bài yêu cầu pipeline phát hiện **1 Critical, 2 High, 2 Medium**. Policy của repo còn chặt hơn: chỉ cần có Critical là block; từ 2 High trở lên cũng block. Report mẫu hiện tại có 15 Critical, 39 High, 48 Medium nên trạng thái là **BLOCKED**.

Sơ đồ logic policy

```
IF critical_count > 0
  THEN BLOCK, owner must fix or request exception
ELSE IF high_count >= 2
  THEN BLOCK
ELSE IF risk_weight > 50
  THEN WARNING, release allowed only with ticket
ELSE
  PASS
```

Điều kiện	Decision	Exception	Approver
Critical bất kỳ	Block	Chỉ cho phép khi false positive đã chứng minh hoặc compensating control tạm thời.	AppSec Lead + Engineering Manager, CISO nếu production impact.
Từ 2 High trở lên	Block	Có expiry date, ticket, owner, rollback plan.	AppSec Lead + Product Owner.
Medium/Low	Warning/Allow	Không cần exception formal nếu đã có backlog và SLA.	Team Lead.

Dashboard / reporting

Nhóm người xem	Họ cần nhìn gì	Trong dashboard/report
CISO/BOD	Security score, số Critical/High, xu hướng block/pass, SLA breach, rủi ro business.	Executive summary, policy status, severity chart, audit log.
Dev team	Finding cụ thể, file/line, root cause, remediation, owner, priority.	Action Center, full_report, triage notes.
Product	Impact tới release, lựa chọn defer/fix, deadline và risk acceptance.	Policy decision, exception workflow, SLA table.

Quy trình xử lý và SLA

Severity	Owner chính	SLA đề xuất	Giao tiếp với Product
Critical	Dev owner + AppSec	24-48 giờ; không release nếu chưa fix/approve exception.	Thông báo release risk ngay, đề xuất hotfix hoặc scope reduction.
High	Dev owner	5 ngày làm việc; block nếu >= 2 hoặc exploitability cao.	Đưa vào sprint hiện tại, thống nhất trade-off deadline.
Medium	Dev team	30 ngày hoặc theo sprint kế tiếp.	Backlog có priority rõ, không che giấu risk.

Task 4. Security of the Pipeline

Threat	Impact	Mitigation đã/đề xuất cài trong pipeline
Secrets leakage	Token/API key bị lộ qua repo hoặc log.	`scan_secret.py` dùng Gitleaks, `.env` không commit, dashboard không yêu cầu nhập secret bắt buộc.
Compromised CI runner	Attacker sửa artifact, đánh cắp biến môi trường, chạy code độc.	Runner container hóa, least privilege volume, artifact audit log, tách secret khỏi source.
Malicious pipeline modification	Bypass scanner hoặc nới policy gate.	Review bắt buộc cho `.gitlab-ci.yml` và `pipeline/`, audit log decision, CODEOWNERS trong CI thật.
Over-permissioned service account	CI token có quyền ghi/xóa quá rộng.	Policy Checkov phát hiện workflow permission rộng; dùng read-only token mặc định, cấp quyền theo job.
Supply-chain attack	Tool/action hoặc dependency bị thay thế độc hại.	Pin version scanner, sinh SBOM, scan dependency bằng Trivy, ưu tiên image/tool nguồn chính thức.

Task 5. Secure-by-Design & Threat Modeling

Task 5 dùng một hệ thống khác với target scan của Task 1-4: **Web App + API + Database + External Payment Service**. Mục tiêu là chứng minh tư duy thiết kế bảo mật trước khi code.

DFD mức cao

```
[Customer Browser]
    | HTTPS + session/JWT
    v
[Web App / Frontend]
    | REST/JSON + access token
    v
[Backend API] ---- SQL/TLS ----> [Database]
    |
    | signed payment request + idempotency key
    v
[External Payment Service]
    |
    v
[Payment Webhook Endpoint] -> verify signature -> update order status
```

Threat mapping

Threat	Design control	CI/CD security check
API/auth: attacker dùng JWT giả, token hết hạn nhưng vẫn accepted hoặc thiếu authorization theo object.	OIDC/JWT verification, short-lived token, RBAC/ABAC, object-level authorization, deny-by-default middleware.	SAST rule cho missing auth middleware, unit/integration test authz, DAST test endpoint unauthenticated.
Payment tampering: sửa amount/order_id hoặc replay webhook.	Server-side price calculation, signed request, webhook signature verification, idempotency key, replay window.	SAST tìm thiếu signature verification, API test replay/idempotency, secret scan cho payment key.
Data disclosure: SQL injection hoặc database exposed/misconfigured.	Parameterized query/ORM safe mode, least privilege DB user, encryption at rest/in transit, private network.	SAST injection rules, SCA database driver CVE scan, IaC scan public exposure/security group.

Liên kết threat model với pipeline

Threat model không dừng ở sơ đồ. Mỗi control phải có một check trong CI/CD: auth control được kiểm bằng SAST và test; payment control được kiểm bằng test chữ ký/webhook; database control được kiểm bằng SAST, SCA và IaC scan. Nhờ vậy thiết kế bảo mật được chuyển thành policy tự động, tránh tình trạng chỉ có tài liệu nhưng không có enforcement.

Checklist nộp bài

Hạng mục	Trạng thái	Đường dẫn
Docker Compose chạy local	Done	`docker-compose.yml`
Pipeline config	Done	`.gitlab-ci.yml`, `pipeline/run_pipeline.sh`
SAST/SCA/IaC/Secret/SBOM reports	Done	`security-results/`
Policy enforcement	Done	`pipeline/policy_engine.py`, `policy_result.json`
Dashboard/reporting	Done	`dashboard/`, `dashboard/data/`
PDF report 3-5 trang	Done	`DevSecOps_CaseStudy_Report.pdf`

Kết luận

AegisFlow đáp ứng yêu cầu thực hành bằng một hệ thống có thể chạy lại local, có artifact rõ ràng, có governance và có phân tích bảo mật của chính pipeline. Các phần giả lập/fallback được giới hạn ở build/test portability, AI triage khi thiếu API key và DAST predictive mode khi không có URL live; các giới hạn này đã được nêu rõ để người chấm phân biệt demo orchestration với production CI hoàn chỉnh.